

Tema 9. El lenguaje WHILE

9.1. Definición formal.

Dados los alfabetos

$$\Sigma_d = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$$

$$\Sigma_w = \{X, :=, 1, +, -, \text{while}, \neq, 0, \text{do}, \text{od}, ;\}$$

Sea $G = (N, T, P, < \text{code} >)$ una gramática con:

$$N = \{ < \text{number} >, < \text{firstdigit} >, < \text{restnumber} >, < \text{digit} >, \\ < \text{code} >, < \text{sentence} >, < \text{assignment} >, < \text{loop} >, < \text{id} > \}$$

$$T = \Sigma_d \cup \Sigma_w$$

$$P = \{ \\ < \text{number} > \rightarrow < \text{firstdigit} > \mid < \text{firstdigit} > < \text{restnumber} >, \\ < \text{firstdigit} > \rightarrow 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9, \\ < \text{digit} > \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9, \\ < \text{restnumber} > \rightarrow < \text{digit} > \mid < \text{digit} > < \text{restnumber} >, \\ < \text{code} > \rightarrow < \text{sentence} > \mid < \text{code} > ; < \text{sentence} >, \\ < \text{sentence} > \rightarrow < \text{assignment} > \mid < \text{loop} >, \\ < \text{id} > \rightarrow X < \text{number} >, \\ < \text{loop} > \rightarrow \text{while } < \text{id} > \neq 0 \text{ do } < \text{code} > \text{ od}, \\ < \text{assignment} > \rightarrow < \text{id} > := 0 \mid < \text{id} > := < \text{id} > \mid \\ & \quad < \text{id} > := < \text{id} > + 1 \mid < \text{id} > := < \text{id} > - 1 \}$$

Definición 9.1.1. Conjunto de códigos WHILE (CODE) (códigos While)

$s \in T^+$ es un código WHILE si y solo si $s \in \text{CODE} = \mathcal{L}(G)$.

Nota. CODE contiene todos los códigos sintácticamente correctos escritos en lenguaje WHILE.

Ejemplo 9.1.1. Código en While. $X2 := X1; \text{while } X2 \neq 0 \text{ do } X1 := X1 + 1; X2 := X2 - 1 \text{ od}$

Definición 9.1.2. Conjuntos de programas While (WHILE) (programas While). Q es un programa While si y solo si $Q \in \text{WHILE} = \{(n, s) \in \mathbb{N} \times \text{CODE}\}$.

Ejemplo 9.1.2. Funcion 'double' en While.

$\text{double} = (1, X2 := X1; \text{while } X2 \neq 0 \text{ do } X1 := X1 + 1; X2 := X2 - 1 \text{ od})$

Ejemplo 9.1.3. funcion 'double' en While-numerado.

$\text{double} = (1, s)$

$s :$

```
1 X2 := X1;
2 while X2 ≠ 0 do
3   X1 := X1 + 1;
4   X2 := X2 - 1
5 od
```

Ejemplo 9.1.4. funcion 'double' en While-etiquetado.

```
double = (1, s)
s :
1 X2 := X1;
2 while X2 ≠ 0 do :6
3   X1 := X1 + 1;
4   X2 := X2 - 1
5 od :2
```

Definición 9.1.3. Tamaño de un código While y un programa While.

$size : CODE \rightarrow \mathbb{N}$

$$size(s) = \begin{cases} 1 & \text{if } \langle assignment \rangle \Rightarrow^* s \\ size(s_1) + 2 & \text{if } s = \text{while } X_i \neq 0 \text{ do } s_1 \text{ od } \wedge \\ & i \in \Sigma_d^+ \wedge s_1 \in CODE \\ size(s_1) + size(s_2) & \text{if } s = s_1; s_2 \wedge s_1, s_2 \in CODE \end{cases}$$

Similarmente, para un programa While (sobrecargando la funcion):

$size : \tilde{WHILE} \rightarrow \mathbb{N}$
 $size((n, s)) = size(s)$

Ejemplo 9.1.5. Tamaño de un programa While para la funcion 'double'.

```
octave> Size('(1, X2:=X1; while X2!=0 do X1:=X1+1; X2:=X2-1 od)')
ans = 5
```

Definición 9.1.4. Funcion de línea.

$line : CODE \times \mathbb{N} \rightarrow T^*$

$$line(s, n) = \begin{cases} s & \text{if } \langle assignment \rangle \Rightarrow^* s \wedge n = 1 \\ \text{while } X_i \neq 0 \text{ do} & \text{if } s = \text{while } X_i \neq 0 \text{ do } s_1 \text{ od } \wedge \\ & i \in \Sigma_d^+ \wedge s_1 \in CODE \wedge n = 1 \\ line(s_1, n - 1) & \text{if } s = \text{while } X_i \neq 0 \text{ do } s_1 \text{ od } \wedge \\ & i \in \Sigma_d^+ \wedge s_1 \in CODE \wedge 1 < n < size(s) \\ \text{od} & \text{if } s = \text{while } X_i \neq 0 \text{ do } s_1 \text{ od } \wedge \\ & i \in \Sigma_d^+ \wedge s_1 \in CODE \wedge n = size(s) \\ line(s_1, n) & \text{if } s = s_1; s_2 \wedge \\ & s_1, s_2 \in CODE \wedge 0 < n \leq size(s_1) \\ line(s_2, n - size(s_1)) & \text{if } s = s_1; s_2 \wedge \\ & s_1, s_2 \in CODE \wedge size(s_1) < n \leq size(s) \\ \varepsilon & \text{otherwise} \end{cases}$$

Definición 9.1.5. Funcion de control de flujo.

$$go : CODE \times \mathbb{N} \rightarrow \mathbb{N}$$

$$go(s, n) = \begin{cases} size(s) + 1 & \text{if } s = \text{while } X_i \neq 0 \text{ do } s_1 \text{ od } \wedge \\ & i \in \Sigma_d^+ \wedge s_1 \in CODE \wedge n = 1 \\ go(s_1, n - 1) + 1 & \text{if } s = \text{while } X_i \neq 0 \text{ do } s_1 \text{ od } \wedge \\ & i \in \Sigma_d^+ \wedge s_1 \in CODE \wedge 1 < n < size(s) \\ 1 & \text{if } s = \text{while } X_i \neq 0 \text{ do } s_1 \text{ od } \wedge \\ & i \in \Sigma_d^+ \wedge s_1 \in CODE \wedge n = size(s) \\ go(s_1, n) & \text{if } s = s_1; s_2 \wedge \\ & s_1, s_2 \in CODE \wedge 0 < n \leq size(s_1) \\ go(s_2, n - size(s_1)) + size(s_1) & \text{if } s = s_1; s_2 \wedge \\ & s_1, s_2 \in CODE \wedge size(s_1) < n \leq size(s) \\ 0 & \text{otherwise} \end{cases}$$

Nota. La funcion 'go' devuelve el numero de línea no consecutivo al que se podría saltar desde una línea dada o cero si desde la línea especificada no se puede saltar a una línea no consecutiva.

Definición 9.1.6. Configuración de un programa While. El conjunto de configuraciones de $Q = (n, s) \in \text{WHILE}$ es $C_Q = \{(m, \underline{x}) \in \mathbb{N}^{p+1} : 1 \leq m \leq size(Q) + 1\}$ donde $p = \max \{n, m \in \Sigma_d^+ : X_m \prec s\}$ (tambien en definiciones adicionales).

Definición 9.1.7. Configuración inicial. Sea $Q = (n, s) \in \text{WHILE}$.

$\underline{c} = (1, \underline{x}) \in C_Q$ es una configuración inicial de Q si y solo si $x_{n+1} = \dots = x_p = 0$.

Definición 9.1.8. Configuración terminal, configuración no-terminal. Sea $Q = (n, s) \in \text{WHILE}$.

$\underline{c} = (m, \underline{x}) \in C_Q$ es una configuración terminal de Q si y solo si $m = size(Q) + 1$.

Por otro lado, podemos decir que es una configuración no-terminal.

Nota. Una configuración no puede ser simultáneamente inicial y terminal.

Definición 9.1.9. Transición directa. Sea $Q = (n, s) \in \text{WHILE}$ y $\underline{c}_1 = (m, \underline{x}), \underline{c}_2 = (t, \underline{z}) \in C_Q$.

Q transita directamente desde $\underline{c}_1$ to $\underline{c}_2$, $\underline{c}_1 \vdash \underline{c}_2$, si y solo si

$$\begin{array}{lll}
line(s, m) = X_i := 0 & \Rightarrow & t = m + 1 \wedge z_i = 0 \quad \wedge z_r = x_r, \forall r \neq i \\
line(s, m) = X_i := X_j & \Rightarrow & t = m + 1 \wedge z_i = x_j \quad \wedge z_r = x_r, \forall r \neq i \\
line(s, m) = X_i := X_{j+1} & \Rightarrow & t = m + 1 \wedge z_i = x_{j+1} \quad \wedge z_r = x_r, \forall r \neq i \\
line(s, m) = X_i := X_{j-1} & \Rightarrow & t = m + 1 \wedge z_i = x_{j-1} \quad \wedge z_r = x_r, \forall r \neq i \\
line(s, m) = \text{while } X_i \neq 0 \text{ do} & \Rightarrow & (x_i \neq 0 \Rightarrow t = m + 1) \quad \wedge \\
& & (x_i = 0 \Rightarrow t = go(s, m)) \quad \wedge \underline{z} = \underline{x} \\
line(s, m) = \text{od} & \Rightarrow & t = go(s, m) \quad \wedge \underline{z} = \underline{x}
\end{array}$$

donde $i, j \in \Sigma_d^+$.

Nota. Las asignaciones actualizan las variables, los encabezados del bucle no alteran las variables y las colas de bucle redirigen a su encabezado.

Definición 9.1.10. Funcion siguiente configuración. Sea $Q = (n, s) \in \text{WHILE}$ (tambien en definiciones adicionales).

$$\begin{aligned}
next_Q &: \mathbb{N}^{p+1} \rightarrow \mathbb{N}^{p+1} \\
next_Q(\underline{c}) &= \begin{cases} \underline{c}' & \text{if } \underline{c}, \underline{c}' \in C_Q \wedge \underline{c} \vdash \underline{c}' \\ \underline{c} & \text{if } \underline{c} \notin C_Q \vee \underline{c} \in C_Q \wedge \nexists \underline{c}' \in C_Q : \underline{c} \vdash \underline{c}' \end{cases}
\end{aligned}$$

Definición 9.1.11. Funcion Calculo

$$\begin{aligned}
cal_Q &: \mathbb{N}^{n+1} \rightarrow \mathbb{N}^{p+1} \\
cal_Q(\underline{x}, t) &= \begin{cases} (1, \underline{x}, \underline{0}) & \text{if } t = 0 \\ next_Q(cal_Q(\underline{x}, t - 1)) & \text{if } t > 0 \end{cases}
\end{aligned}$$

donde $t \in \mathbb{N}, \underline{x} \in \mathbb{N}^n \wedge \underline{0} \in \mathbb{N}^{p-n}$.

Definición 9.1.12. Funcion de complejidad temporal.

$$\begin{aligned}
T_Q &: \mathbb{N}^n \rightarrow \mathbb{N} \\
T_Q(\underline{x}) &= \begin{cases} minimum(A) & \text{if } A \neq \emptyset \\ \uparrow & \text{if } A = \emptyset \end{cases}
\end{aligned}$$

donde $A = \{t \in \mathbb{N} : \pi_1^{p+1}(cal_Q(\underline{x}, t)) = size(Q) + 1\}$

Definición 9.1.13. Funcion calculada.

$$\begin{aligned}
F_Q &: \mathbb{N}^n \rightarrow \mathbb{N} \\
F_Q(\underline{x}) &= \pi_2^{p+1}(cal_Q(\underline{x}, T_Q(\underline{x})))
\end{aligned}$$

Ejemplo 9.1.6. funciones $next_Q$ y cal_Q

```

octave> Next('(1, X2:=X1; while X2!=0 do X1:=X1+1; X2:=X2-1 od)',
[1,3,0])
ans =
2 3 3

octave> Cal('(1, X2:=X1; while X2!=0 do X1:=X1+1; X2:=X2-1 od)', 5,
1)
ans =
2 5 5

```

9.2. Computabilidad, decidibilidad, numerabilidad

Definición 9.2.1. Funcion While-calculable.

f es una funcion While-calculable si y solo si $\exists Q \in \text{WHILE} : F_Q = f$.

$F(\text{WHILE})$ es el conjunto de todas las funciones While-calculables.

Definición 9.2.2. Funciones While-calculables totales.

$\text{T-WHILE} = \{f \in F(\text{WHILE}) : f \text{ is a total function}\}.$

Definición 9.2.3. Predicados While-decidibles.

$\text{PRED}(\text{T-WHILE}) = \{P_f : f \in \text{T-WHILE}\}.$

Definición 9.2.4. Predicados While-numerables.

$\text{PRED}(\text{WHILE}) = \{P_f : f \in F(\text{WHILE})\}.$

Definición 9.2.5. Conjuntos While-decidibles.

$\text{DEC}(\text{WHILE}) = \{V_P : P \in \text{PRED}(\text{T-WHILE})\}.$

Definición 9.2.6. Conjuntos While-numerables.

$\text{ENU}(\text{WHILE}) = \{V_P : P \in \text{PRED}(\text{WHILE})\}.$

Equivalencias entre funciones, predicados y conjuntos:

F(MT)	REC	F(WHILE)
DEC(MT)	DEC	DEC(WHILE)
ENU(MT)	ENU	ENU(WHILE)
T-MT	TREC	T-WHILE
PRED(T-MT)	PRED(TREC)	PRED(T-WHILE)
PRED(MT)	PRED(REC)	PRED(WHILE)